Popular destinations for consuming metrics include the following:

- Frontends, such as low-level/debug offerings, including the Prometheus UI or PromLens (https://github.com/promlabs/promlens-public)
- Universal data sources frontends, such as Grafana (https://github.com/grafana/grafana)
- Commercial or integrated offerings, including CloudWatch, New Relic, Datadog, and so on
- Object storage, such as an S3 bucket, for later analysis

With this, we've reached the end of our metrics cost and benefit discussion, so let's move on to metrics use cases for observability.

2.4.4 Observability with metrics

What can we do with metrics, and how is what we just saw related to observability? In a DevOps or SRE role, we can do the following:

- *Establish baselines for signals, based on metrics*—Further, you can define anomalies and expectations via alerts and get notifications if certain values are exceeded (this is more of a traditional monitoring use case for metrics, since the questions are already known up front).
- *Track metrics over time and study histograms*—This can help for capacity planning or scalability conversations.
- *Define operational and tactical goals*—Usually, this is called *service level objective* (SLO) (e.g., "serve less than 12% errors, over a period of a week"). To assess whether the goal has been met, one uses a measurement—usually called a *service level indicator* (SLI)—such as the error rate (ratio of errors to overall requests). Chapter 10 is dedicated to SLOs, so feel free to skip ahead now if you want to learn more.

Now, let's take a look at how to deal with traces.

2.5 Traces

When you hear the term *traces*, it can mean that you have to disambiguate things first—that is, be sure to ask what someone really means when referring to traces. In general, we can differentiate between two cases—local or distributed:

- In the case of local "traces," the more widely used and established term is *profiles*. We use *tracing* and *sampling* (regarding a profile) interchangeably. This is related to capturing certain characteristics of the execution of a process in the context of a single machine, such as kernel-level events or user space function calls along with execution time and resource usage, including CPU and memory, such as in the case of using `strace` to see syscalls. If you go a step further